CS 25000 Lab 10 – Processor Design

Work in pairs or a triple if there is an odd number present in your lab session.
Upload by the end of your lab session to Gradescope.

Instructions on how to process the Word file forthis assignment.

- Download this file.
- Delete nothing from this Word file.
- Edit this file to add your typewritten answers to each question.
- Ensure that your answer fits on the same page as its question. If you change the pagination of this file or if your complete answer to a question does not fit on the page with that question, then you may receive a lower score.
- When your answer includes a hand-drawn diagram make it clear, large, and legible.
- Export your completed Word file to PDF.
- Upload your PDF file to its corresponding assignment portal in Gradescope.com. You may upload multiple times. Your final upload will be scored. As desired, use the download capability to check your upload.
- **Upload before leaving your lab session.**
- Max score = 20 points; equal weight per question.

- **Why should your answer be on the same page with its question?**
  Answer: Gradescope has been programmed to expect that your PDF file will have exactly 10 pages, each containing a question and your entire answer. This allows Gradescope to automatically display each answer for scoring.

1. Consider the following instruction:

   Instruction: AND Rd, Rn, Rm
   Interpretation: Reg[Rd] = Reg[Rn] AND Reg[Rm]

   Which resources (blocks) in zyBook Figure 4.3.4 produce no output for this instruction? Which resources produce output that is not used?

   All of the resource blocks produce and output.
   Sign extend and data memory outputs are not used.

2. Consider the following instruction mix:

R-type	I-Type	LDUR	STUR	CBZ	B
24%	28%	25%	10%	11%	2%

(a) What fraction of all instructions use data memory?

35/100, LDUR and STUR

(b) What fraction of all instructions use Sign Extend?

76/100, no I-Type

3. The LEGv8 processor fetches the following instruction word: 0xf8014062. For context: The encoded instruction is STUR X2, [X3, 0x14].
    a. What are the outputs of the sign-extend and the shift left 2 units (see Figure 4.4.10) for this instruction word?

       0000 0000 0000 0014
       0000 0000 0000 0050

4. Assume that instructions executed by aLEGv8 processor are broken down as follows:

ALU/Logic	Jump/Branch	LDUR	STUR
45%	20%	20%	15%

a. Assuming no stalls or hazards, what is the percent of clock cycles in which the data memory is utilized?

35%, LDUR and STUR

b. Assuming that there are no stalls or hazards, what is the utilization of the write-register port of the register file?

65% ALU/logic and Jump/Branch

5. What is the minimum number of cycles needed to completely execute n instructions on a CPU with a k stage pipeline? Justify your formula.

   n + k - 1

   n instructions will require at least n cycles, plus the k cycles of the last instruction needing to finish executing, but we subtract 1 so we don't count the first stage of the last instruction twice.

6. Assume that X1 is initialized to 11 and X2 is initialized to 22. Suppose you executed the code snippet below on a version of the classic 5-stageLEGv8 pipeline from Section 4.5 that has no forwarding and does not handle data hazards (i.e., the assembly language programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). The snippet has no inserted NOPs, so which results of the code snippet will be incorrect and which will be correct? Assume the register file is written at the beginning of a clock cycle and read at the end of a cycle. Therefore, the ID stage can use the result of a WB occurring during the same cycle.

```
ADDI X1, X2, #5
ADD  X3, X1, X2
ADDI X4, X1, #15
ADD  X5, X1, X1
```

X3 will have the incorrect value, as X1 will not be ready when ADD X3, X1, X2 tries to grab it's value. The same is true for X4.
All other results will be correct.

7. Add NOPs to the code snippet below so that it will compute only correct results on the pipeline that does not handle data hazards.

```
                ADDI X1, X2, #5
NOP
NOP
                ADD  X3, X1, X2
                ADDI X4, X1, #15
NOP
                ADD  X5, X3, X2
```

8. Examine the accuracy of various branch predictors for the following repeating patten (e.g., in a loop) of five branch outcomes: T, NT, T, T, NT.
    a. Let the states of the 2-bit predictor be numbered 0, 1, 2, and 3 where the states are String Not Taken, Weak Not Taken, Weak Taken, and Strong Taken predictions, respectively. What is the accuracy of the 2-bit predictor for the first four branches in this pattern, assuming the predictor starts off in the bottom left state (predict not taken) from Figure 4.8.2? Fill in the table with your answer.

Outcomes	Predictor state at time of prediction	Correct or Incorrect	Accuracy
T	Not Taken	Incorrect	0%
NT	Weak Not Taken	Correct	50%
T	Not Taken	Incorrect	33%
T	Weak Not Taken	Incorrect	25%

    b. Let the states of the 2-bit predictor be numbered 0, 1, 2, and 3 where the states are String Not Taken, Weak Not Taken, Weak Taken, and Strong Taken predictions, respectively. What is the accuracy of the 2-bit predictor if the T, NT, T, T, NT pattern is repeated forever? Fill in the table with your answer. Note that this predictor must "warm up" to its best accuracy prediction.

Outcomes	Predictor state at time of prediction	Correct or Incorrect (in steady state)	Accuracy
T	Weak Taken	Correct	100%
NT	Strong Taken	Incorrect	50%
T	Weak Taken	Correct	66%
T	Strong Taken	Correct	75%
NT	Strong Taken	Incorrect	60%

9. To improve pipeline CPI, what does a compiler do as the step following loop unrolling?

Dynamic pipeline scheduling, including potential instruction reordering

10. Besides improving the chances of a good instruction schedule, what other way does loop unrolling reduce the total number of clock cycles needed to execute a for loop?

    You can reduce the number of iterations (overhead) by increasing the step size and adding more to the loop body.

```
for (int i = 0; i < 1000; i++) {
        doSomething(i);
}

vs

for (int i = 0; i < 1000; i+=5) {
        doSomething(i);
        doSomething(i+1);
        doSomething(i+2);
        doSomething(i+3);
        doSomething(i+4);
}
```